Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Why quantization, and the forward pass during training

Why quantization.

- Billions of weights, each 32 or 16 bits.
- Too big for a phone, slow, costly in energy.
- Quantization: 4, 2 or 1 bit per weight.

Training with quantization.

- Training keeps the float weights x.
- Forward pass uses quantized $\hat{x}$; backward pass updates the float x.
- Inference keeps only $\hat{x}$ (stored as q plus s_f, b_f per block).

The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid $\{0, \ldots, 2^k - 1\}$, with $s_f = \dfrac{x_{\max} - x_{\min}}{2^k - 1}$, $b_f = x_{\min}$.
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^\top \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\mathrm{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Round as before in the forward pass, but treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad keeps the value in the forward pass and has zero derivative in the backward pass.

Forward vs. backward.

$$\text{forward:}\quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:}\quad \frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \quad \nabla_x L = \nabla_{\hat{x}} L.$$

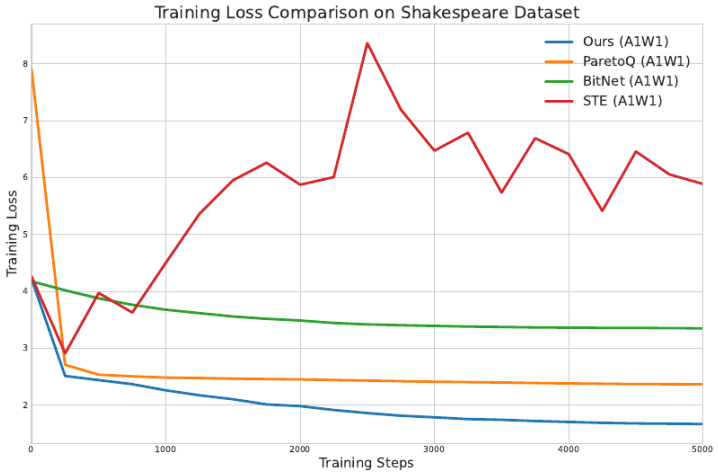
The gradient reaching x is no longer zero, so training works.

The blind spot.

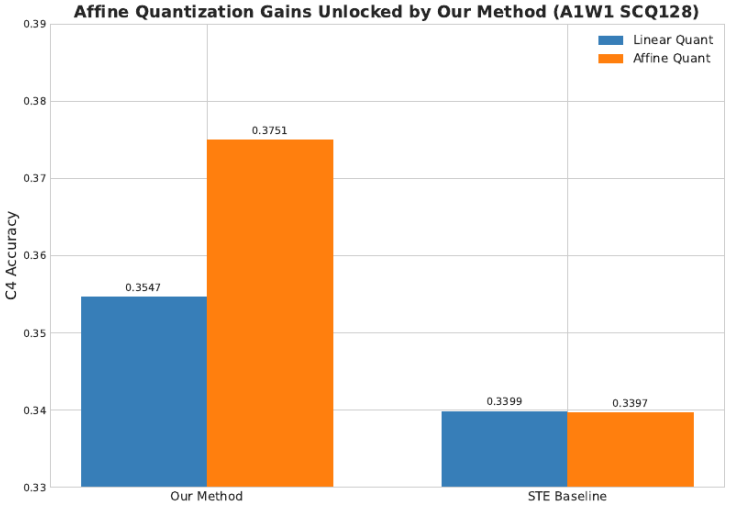
- The forward pass shifts every entry by $s_f\delta_i$, but the Jacobian is I, exactly as with no quantization. δ does not appear in it.
- So earlier layers get no signal about how rounding hurts the loss, and cannot learn to be robust to it.
- At 8 bits $s_f\delta$ is tiny and this is harmless. At 1–2 bits it is as large as the values themselves: STE diverges.

Reference: Y. Bengio, N. Léonard, A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

STE at 1 bit: results from the paper



Training loss, 1-bit weights and activations (A1W1), small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.



Accuracy at A1W1, linear vs. affine quantization. Under STE affine gives no gain; with the paper's method it does.

Source: Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity," ICLR 2026, Fig. 1.